

Ports and applications

Goal of the chapter

In this chapter you will start coding in Java to develop networked applications.

1 A client-server Java program

Goal

We would like to create a Java program that connects to a web server, say google.com. So these are the steps:

- Create a socket to the web server listening on port 80.
- Obtain a `PrintStream` to the server and send the request `GET PATH HTTP/1.0`, where `PATH` is the requested resource on the server. For example, if we wanted to open the root of a web site then the path would be `/`.
- Obtain an `InputStream` to the server, wrap it with a `BufferedReader` and read the response line-by-line.

Accessing a remote computer in Java can be done using the library `java.net`, in particular the class `java.net.Socket`. For example, the following code opens a connection to a server:

```
1 Socket socket = new Socket( server, port );
```

Once our socket instance is connected to the server we can start obtaining input and output streams to the sever. Input streams are used to read data from the server while output streams are used to write data to the server. We can execute the following methods to obtain input and output streams:

```
1 InputStream in = socket.getInputStream();  
2 OutputStream out = socket.getOutputStream();
```

1.0.1 Creating an Eclipse project

- In Eclipse, create a Java project.
- Let us call this project “simplesocketclient”
- The next step is to create a Java class that will contain our code. We do so by right clicking again on the project, and telling Eclipse to create a Java class.
 - For this example use the package `edu.deakin.sit219.socket`
 - Use the class name `SimpleSocketClientExample`
 - Click finish.
 - Add the following code.

```

1 package edu.deakin.sit218.socket;
2
3 import java.io.BufferedReader;
4 import java.io.InputStreamReader;
5 import java.io.PrintStream;
6 import java.net.Socket;
7
8 public class SimpleSocketClientExample {
9     public static void main( String[] args )
10    {
11        String server = "www.google.com";
12        String path = "/";
13
14        System.out.println( "Loading contents of URL: " +
server );
15
16        try
17        {
18            // Connect to the server
19            Socket socket = new Socket( server, 80 );
20
21            // Create input and output streams to read from
and write to the
22            //server
23            PrintStream out = new PrintStream( socket.
getOutputStream() );
24            BufferedReader in = new BufferedReader( new
InputStreamReader(
25            socket.getInputStream() ) );
26
27            // Follow the HTTP protocol of GET <path> HTTP
/1.0 followed by an
28            //empty line
29            out.println( "GET " + path + " HTTP/1.0" );
30            out.println();
31
32            // Read data from the server until we finish
reading the document
33            String line = in.readLine();
34            while( line != null )
35            {
36                System.out.println( line );
37                line = in.readLine();
38            }
39
40            // Close our streams
41            in.close();
42            out.close();
43            socket.close();
44        }
45        catch( Exception e )
46        {
47            e.printStackTrace();
48        }
49    }
50 }
51
52
53

```

Naming a package in Java

- Package names are written in all lower case to avoid conflict with the names of classes or interfaces.
- Companies use their reversed Internet domain name to begin their package names—for example, `com.example.mypackage` for a package named `mypackage` created by a programmer at `example.com`. In our case, we are using the `edu` top level domain.
- Name collisions that occur within a single company need to be handled by convention within that company, perhaps by including the region or the project name after the company name (for example, `com.example.region.mypackage`).
- Packages in the Java language itself begin with `java.` or `javax.`

When you retrieve a web page from a web server, such as `www.google.com`, the HTTP client uses DNS servers to find the server's address: it starts by asking the top-level domain server for the `com` domain where the authoritative domain-name server is for `www.google.com`. Then it asks that domain-name server for the IP address (or addresses) for `www.google.com`. Next, it opens a socket to that server on port 80. (Or, if you want to define a different port, you can do so by adding a colon followed by the port number, for example: `:8080`.) Finally, the HTTP client executes the specified HTTP method, such as GET, POST, PUT, DELETE, HEAD, or OPTIONS. Each method has its own syntax. As shown in the above code snips, the GET method requires a path followed by HTTP/version number and an empty line. If we wanted to add HTTP headers we could have done so before entering the new line.

In the code above, we retrieved an `OutputStream` and wrapped it in a `PrintStream` so that we could more easily execute our text-based commands. Our code obtained an `InputStream`, wrapped that in an `InputStreamReader`, which converted it to a `Reader`, and then wrapped that in a `BufferedReader`. We used the `PrintStream` to execute our GET method and then used the `BufferedReader` to read the response line-by-line until we received a null response, indicating that the socket had been closed.